Computational neuroscience mini project

Gaétan Bouthors, Marieke Vaske

May 2026

Introduction

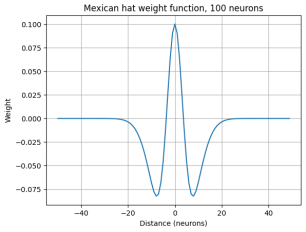
This project investigates the computational mechanisms of spatial navigation by simulating grid cells within a Continuous Attractor Network model. The core problem centers on the challenge of path integration: how a neural population can maintain and update a stable representation of an animal's position using only self-motion velocity cues. We model in 1D, and then 2D an array of neurons that interact with each other following a Mexican-hat kernel, creating stable bumps, that can then reproduce movement as the network is influenced by velocity inputs.

Approach

We wrote all the code from scratch on the basis of experience implementing similar problems in other courses. The instructions were quite explicit on what we needed to do for each section. We used the LLM Claude to help plot solutions and to help debug. For debugging, we prompted with the problematic code, the part of output that was unexpected and what we expected to see, or by asking it to list possible reasons a particular unexpected trend was visible in the output. We chose to make Jupyter Notebooks as they allow to easily test different parts of the code and generate each figure in its own section.

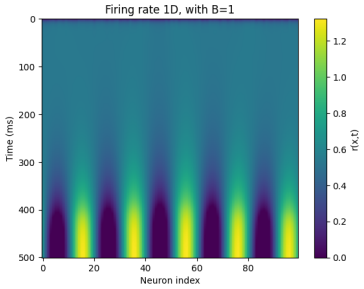
Exercise 0. Ring Attractors in 1D

0.1 & 0.2



0.3 This kernel represents local excitation and lateral (or surround) inhibition. In cortical networks, neurons that are close to each other tend to have mutually excitatory connections, while neurons slightly further away inhibit each other.

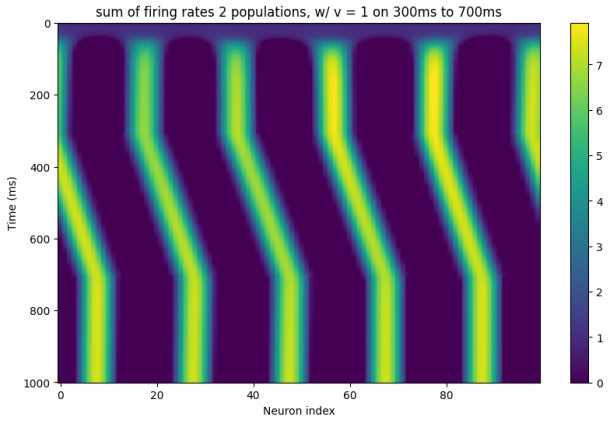
0.4



0.5 The network quickly evolves into stable, periodic, localized bumps of firing activity. When using different random seeds for initialization, the eventual locations (phases) of these bumps can shift, and the time it takes to reach an equilibrium state differs slightly, but the periodicity (distance between bumps) and the shape of the bumps remain identical. The fundamental behavior does not depend on the initialization.

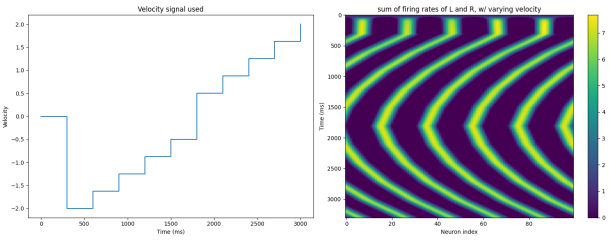
Exercise 1. 1D Velocity Integration

1.1

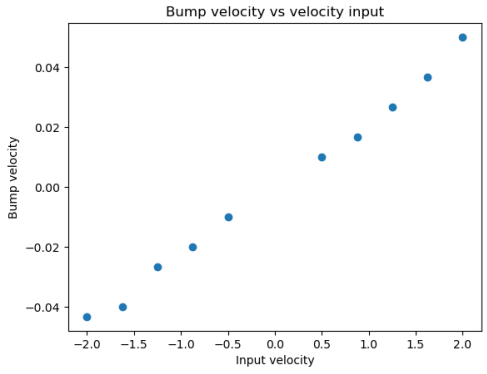


1.2 The recurrent weights from the Right (R) population are shifted slightly to the right, and the Left (L) population to the left. If velocity $v > 0$, the external input B_R increases while B_L decreases. This causes the R population to fire more strongly than the L population. Because R's outgoing weights are shifted rightward, the total recurrent excitation is skewed to the right of the current bump center, pulling the bump to the right.

1.3 When a negative velocity is applied, the bump moves left, and when a positive velocity is applied, the bump moves right. The magnitude of the velocity results in the bump moving faster, hence why we observe the bumps move fast to the left, then slow down, start moving to the right and accelerate.

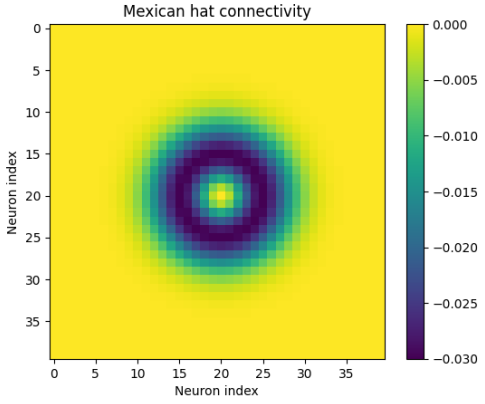


1.4 We track one of the bumps and compute the distance over which the maximum firing rate has been propagated. To do this, we use the "np.argmax" function and find the peak value in the vicinity of the previously recorded peak value at regular intervals averaging the total distance over the time interval during which the velocity was used. When plotted against the input velocity, it yields a highly linear relationship, confirming the network acts as a velocity integrator.

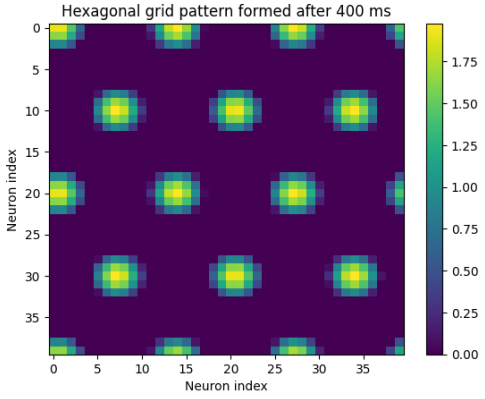


Exercise 2. Extending to 2D Grid Cells

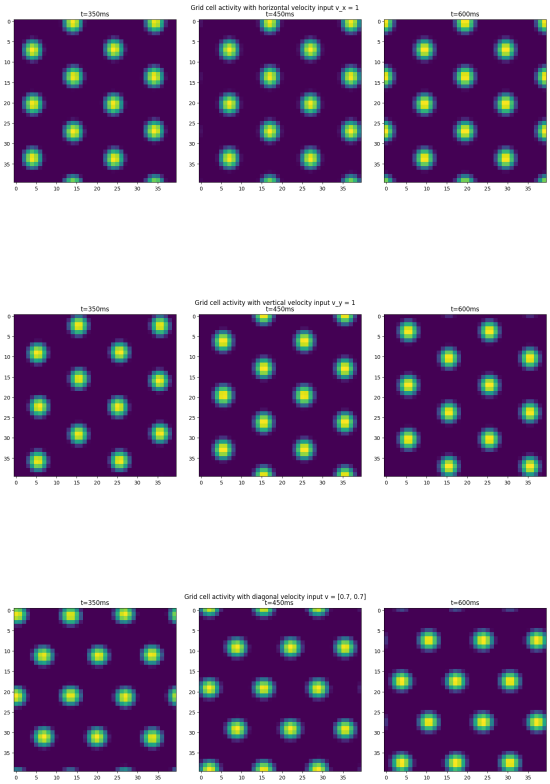
2.1



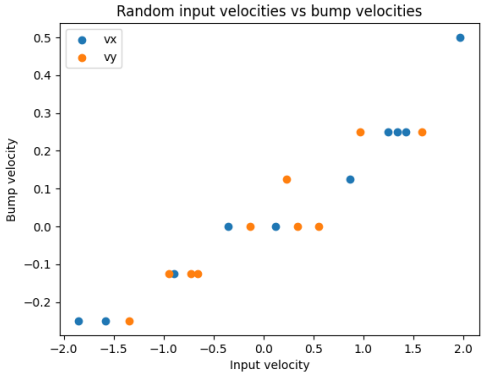
2.2 After the activity stabilizes, we obtain a hexagonal lattice pattern.



2.3 The entire lattice shifts coherently in the exact direction of the velocity vector. Because of the periodic boundaries, bumps that fall off one edge seamlessly reappear on the opposite edge.

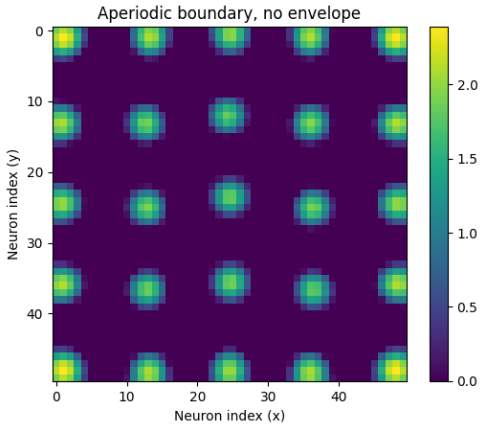


2.4 We can see in the animation the bumps move in changing directions. Calculating the velocities and plotting them against the true velocity, we once again obtain a near-linear relationship.

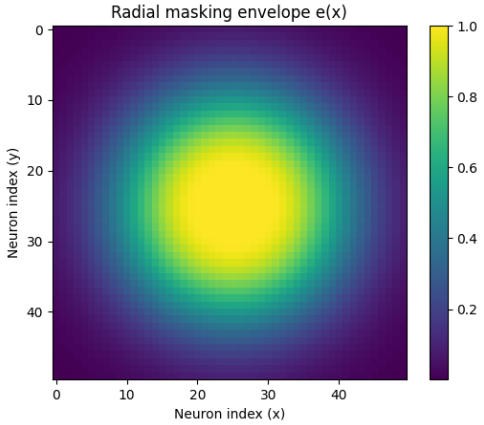


Exercise 3. Aperiodic Boundaries

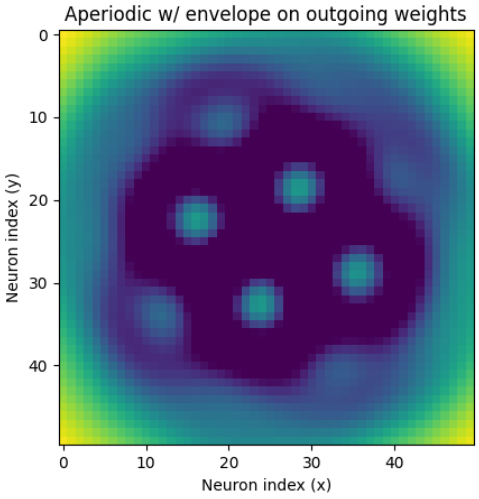
3.1 This time the activity stabilizes into a rectangular lattice pattern.



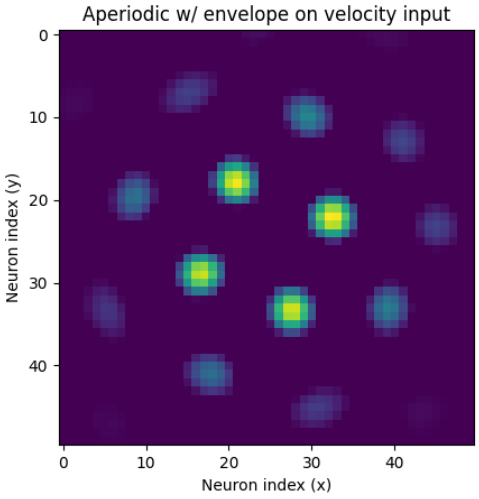
3.2



3.3 Scaling the firing rates by the envelope e(x) before convolution dampens the recurrent drive of edge neurons. This leads to very high activity along the edges, where the external input dominates the reccurent input, but these neurons also don't have strong outgoing effect anyways. In the center, where excitation and lateral inhibition function correctly, the desired hexagonal lattice pattern emerges.



3.4 Multiplying the feedforward baseline inputs (B_N, B_S, B_E, B_W) by the envelope recovers the hexagonal grid in the center by lowering the baseline excitability of boundary neurons. This results in the firing rate fading along the borders.



3.5 Both approaches allow the pattern to emerge in the center, one having very high firing rate along the border, and the other very low. The first approach appears less effective however, given there are more artefacts along the edges resulting in a smaller grid, and possibly more disruption when velocity is introduced. The second approach generates just the grid, with the intensity fading as we approach the edges. Also, approach 3.4 seems more biologically plausible as it suggests that the grid is formed where the relevant sensory and velocity inputs are strongest, and it simply tapers off where those inputs are absent, rather than requiring a precise scaling of every outgoing synapse at a physical edge.

3.6 The bump and input velocities still have a roughly linear relation, but it seems there is a bit more variance. While observing the animation, there seems to be some amount of rotation: bumps don't seem to have exactly the same speed as each other, as the center is more reactive than the borders, since the bumps have higher firing rate. The movement in the periodic model was much more straight and coherent as it was space invariant.

Exercise 4.2 : Mapping Between Code and Burak & Fiete (2009)

We systematically compare each component of our implementation to the model described in Burak & Fiete (2009).

Neural dynamics. The paper's update rule and ours are different in principle: in the paper f acts on $(Ws + B)$, whereas in our code f acts on s alone and B enters additively outside f. Both formulations produce qualitatively identical grid-cell dynamics.

Synaptic weight kernel. The paper defines a centre-surround profile that is equivalent to our parametrisation with the mapping is $\sigma_{\text{exc}} = \gamma^{-1/2}$ and $\sigma_{\text{inh}} = \beta^{-1/2}$. With the paper's values one obtains $\sigma_{\text{exc}} \approx 7.33$ and $\sigma_{\text{inh}} \approx 7.51$, which are wider than our $\sigma_{\text{exc}} = 4.8$, $\sigma_{\text{inh}} = 5.0$. Our narrower kernels are a rescaling to match the smaller network ($M = 40$ versus $N = 128$) while preserving the ratio $\sigma_{\text{exc}}/\sigma_{\text{inh}} \approx 0.96$.

Population structure and weight shift. The paper arranges all four preferred-direction neurons $(\text{N}, \text{S}, \text{E}, \text{W})$ in a single interleaved sheet: every 2×2 block contains exactly one neuron of each type. The kernel is shifted in the preferred direction of the source neuron.

Our code instead stores four separate $M \times M$ arrays, one per population. The shifted kernels `wN`, `wS`, `wE`, `wW` are pre-computed by rolling the base kernel by $\ell = 1$ along each axis. Each population is then convolved with its own shifted kernel and the four recurrent inputs are summed. This is mathematically equivalent to the paper's scheme, just organized differently.

Network size and numerical time step. The paper uses $N = 128 \times 128 = 16{,}384$ neurons per population with a time step $\Delta t = 0.5$ ms. Our implementation uses $M \times M = 50 \times 50 = 2{,}500$ neurons and $\Delta t = 1$ ms.

External (velocity) input. The paper writes the input to neuron i with preferred direction $\hat{\mathbf{e}}_{\theta_i}$ as

$$B_i = A(\mathbf{x}_i)\big(1 + \alpha\, \hat{\mathbf{e}}_{\theta_i} \cdot \mathbf{v}\big). \tag{1}$$

Our code implements the same expression for each of the four populations:

$$B_{\text{N}} = B_0(1 + \alpha v_y), \qquad B_{\text{S}} = B_0(1 - \alpha v_y), \tag{2}$$

$$B_{\text{E}} = B_0(1 + \alpha v_x), \qquad B_{\text{W}} = B_0(1 - \alpha v_x), \tag{3}$$

which is identical to the paper's formula with $A(\mathbf{x}_i) = B_0 = 1$ (periodic case) and the dot product resolved per population. The gain factor $\alpha = 0.1$ in our code matches the paper's parameter. In the aperiodic case, $A(\mathbf{x}_i)$ scales the feedforward inputs for distances beyond a certain radius (a fifth of network radius) the same way we scale by e(x). In both cases, the scaling is a gaussian function of the distance beyond a fade

radius, and mathematically equivalent up to parameters. The paper has no counterpart to the method in 3.3.

Conclusion

This project demonstrates that while Continuous Attractor Networks effectively model grid cell path integration, their stability is highly sensitive to boundary conditions, and it can be challenging to model aperiodic networks that better simulate biological reality. Ultimately, this work reinforces the CAN model's validity as a mechanistic explanation for the brain's internal coordinate framework.